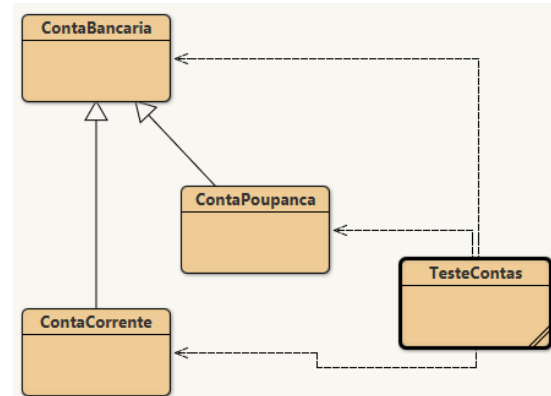


Sobrescrita de Métodos

A sobrescrita de métodos em Java é um conceito fundamental da programação orientada a objetos, que permite a uma subclasse fornecer uma implementação específica de um método que já está sendo usado por sua superclasse. Isso é útil quando você deseja que um método se comporte de forma diferente em diferentes classes, mantendo a mesma assinatura.

Ao lado, temos um exemplo com as classes **ContaCorrente** e **ContaPoupanca**, ambas sendo subclasses de uma classe **ContaBancaria**. Neste caso, o polimorfismo permite tratar objetos dessas classes de forma genérica.



Classe ContaBancaria: O atributo saldo é protegido para que possa ser acessado pelas subclasses, mas não diretamente por outras classes fora do pacote. o atributo saldo não deve ser acessado diretamente por outras classes fora do pacote (por exemplo, em outras classes que não sejam subclasses de ContaBancaria) porque isso violaria o princípio de encapsulamento. O encapsulamento é um conceito que busca proteger os detalhes de implementação de uma classe e expor apenas uma interface pública para interagir com ela.

```
public class ContaBancaria {
    // Atributo protegido para armazenar o saldo da conta
    protected double saldo;

    // Construtor que recebe um saldo inicial e o atribui ao saldo da conta
    public ContaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    // Método para depositar um valor na conta, adicionando ao saldo
    public void depositar(double valor) {
        this.saldo += valor;
    }

    // Método para sacar um valor da conta, subtraindo do saldo
    // Retorna true se o saque foi bem-sucedido, false caso contrário
    public boolean sacar(double valor) {
        if (this.saldo >= valor) {
            this.saldo -= valor;
            return true;
        }
        return false;
    }

    // Método para obter o saldo da conta
    public double getSaldo() {
        return this.saldo;
    }

    // Método para imprimir o saldo da conta
    public void imprimirSaldo() {
        System.out.println("Saldo da conta: " + this.saldo);
    }
}
```

Classe **ContaCorrente**

O método `imprimirSaldo` é sobrescrito para imprimir o saldo da conta corrente.

```
public class ContaCorrente extends ContaBancaria {  
    // Atributo privado para armazenar a taxa de operação da conta corrente  
    private double taxaDeOperacao;  
  
    // Construtor que recebe um saldo inicial e uma taxa de operação  
    // Chama o construtor da superclasse para inicializar o saldo  
    public ContaCorrente(double saldoInicial, double taxaDeOperacao) {  
        super(saldoInicial);  
        this.taxaDeOperacao = taxaDeOperacao;  
    }  
  
    // Sobrescrita do método da superclasse para imprimir o saldo da conta corrente  
    @Override  
    public void imprimirSaldo() {  
        System.out.println("Saldo da conta corrente: " + this.saldo);  
    }  
}
```

Classe **ContaPoupanca**

O método `imprimirSaldo` é sobrescrito para imprimir o saldo da conta poupança.

```
public class ContaPoupanca extends ContaBancaria {  
    // Atributo privado para armazenar a taxa de juros da conta poupança  
    private double taxaDeJuros;  
  
    // Construtor que recebe um saldo inicial e uma taxa de juros  
    // Chama o construtor da superclasse para inicializar o saldo  
    public ContaPoupanca(double saldoInicial, double taxaDeJuros) {  
        super(saldoInicial);  
        this.taxaDeJuros = taxaDeJuros;  
    }  
  
    // Sobrescrita do método da superclasse para imprimir o saldo da conta poupança  
    @Override  
    public void imprimirSaldo() {  
        System.out.println("Saldo da conta poupança: " + this.saldo);  
    }  
}
```

No método `main()`, quando chamamos `conta1.imprimirSaldo()` e `conta2.imprimirSaldo()`, estamos chamando o método `imprimirSaldo()` das subclasses **ContaCorrente** e **ContaPoupanca**, respectivamente, devido ao polimorfismo.

Isso significa que o Java determina qual método chamar com base no tipo de objeto em tempo de execução, permitindo que você trate objetos de subclasses de forma genérica como objetos da superclasse. Essa é uma das vantagens da orientação a objetos e do princípio do polimorfismo.

```
public class TesteContas {  
    public static void main(String[] args) {  
        // Cria uma conta corrente com saldo inicial de 1000 e taxa de operação de 10  
        ContaBancaria conta1 = new ContaCorrente(1000, 10);  
        // Cria uma conta poupança com saldo inicial de 2000 e taxa de juros de 0.05  
        ContaBancaria conta2 = new ContaPoupanca(2000, 0.05);  
  
        // Deposita 500 na conta corrente  
        conta1.depositar(500);  
        // Deposita 1000 na conta poupança  
        conta2.depositar(1000);  
  
        // Imprime o saldo da conta corrente  
        // Deve imprimir o saldo atualizado após o depósito  
        conta1.imprimirSaldo();  
  
        // Imprime o saldo da conta poupança  
        // Deve imprimir o saldo atualizado após o depósito  
        conta2.imprimirSaldo();  
    }  
}
```